

Vision Editor

Faculty Faculty of Computers and Artificial Intelligence
University October 6 University
Department Computer Science
Course Image Processing
Academic Year 2025 / 2026

Student Name	Student Code
Assem Atya Abdelwhab	23017833
Ahmed Mostafa Mahmoud	23013835
Mostafa Tarek Mahmoud	23012714
Ahmed Yasser Mohammed Sadik	23018414
Mustafa Elsebaey Mohamed	23017722

Project Title: "Vision Editor" – Advanced Image Enhancement & Analysis

Project Objective

To build a functional GUI application that allows users to perform image processing operations, visualize data distributions via histograms, and evaluate the impact of preprocessing on Computer Vision (CV) task accuracy.



1. Introduction

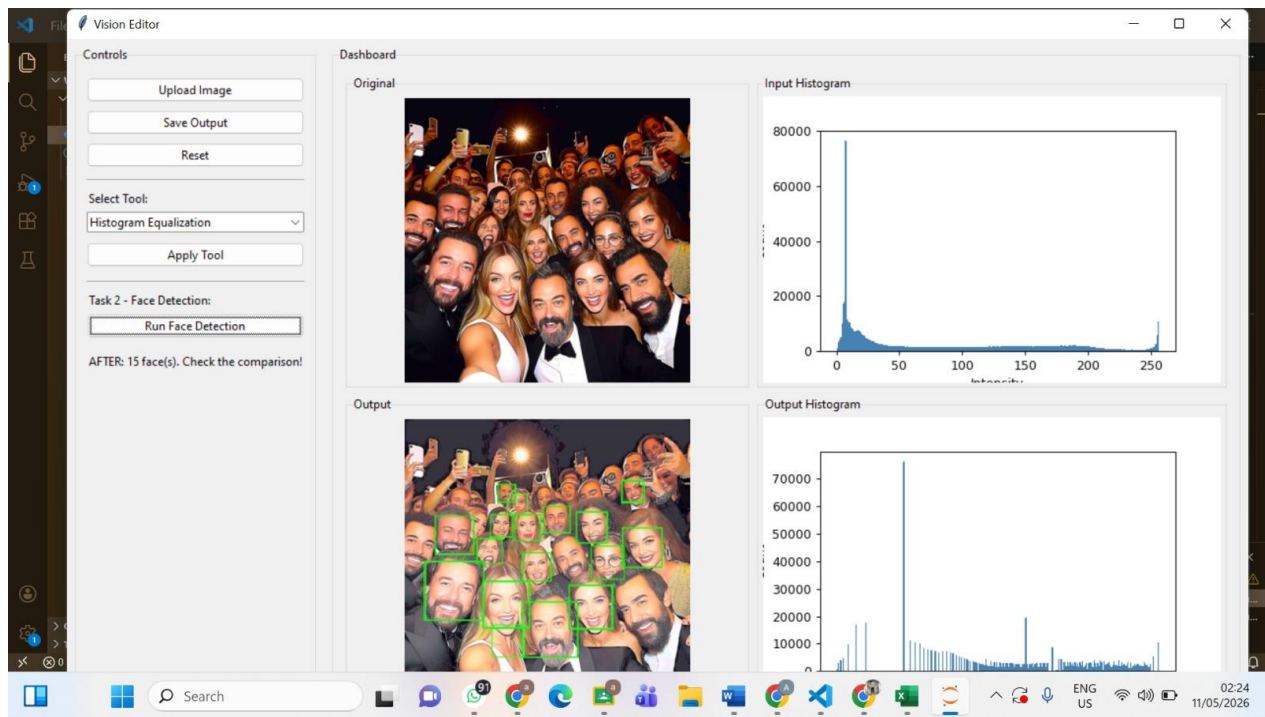
1.1 Domain of Problem

Digital image processing is a fundamental discipline in computer vision and artificial intelligence that focuses on manipulating and analyzing visual data to extract meaningful information or improve image quality. In real-world applications, images are often captured under suboptimal conditions — poor lighting, noise, low contrast, or geometric distortions — leading to degraded performance in downstream computer vision tasks.

This project addresses the challenge of building a practical, interactive tool that empowers users to apply a range of image processing techniques and immediately observe their effect on both image quality and computer vision task accuracy. The domain spans digital image enhancement, histogram analysis, geometric transformations, and automated CV task evaluation.

1.2 Screenshot of the Application

The following screenshot shows the Vision Editor GUI in action — running face detection on a real image after applying Histogram Equalization:



1.3 Image Processing Techniques Covered

The following categories of image processing techniques are implemented in this project:

- **Point Operations: Brightness and Contrast adjustment using linear scaling ($\alpha \cdot \text{pixel} + \beta$).**

These operations modify individual pixel intensities without considering spatial context, making them simple yet effective for basic enhancement.

- **Geometric Transformations: Rotation and Zooming.**

Rotation uses affine transformation matrices; zooming is implemented via two interpolation methods — Nearest Neighbor and Bilinear — for comparison of quality vs speed.

- **Histogram-Based Enhancement: Histogram Equalization and CLAHE.**

Histogram Equalization redistributes pixel intensities globally to improve contrast. CLAHE (Contrast Limited Adaptive Histogram Equalization) applies equalization locally in tiles, preventing over-amplification of noise.

- **Gamma Correction: Adjusts luminance using a power-law transform.**

Values $\gamma > 1$ brighten dark images; $\gamma < 1$ darkens bright images.

- **Spatial Filtering: Gaussian Blur and Median Filter.**

Gaussian Blur uses a Gaussian kernel for smoothing (reduces noise while slightly blurring edges). Median Filter replaces each pixel with the median of its neighbors, effective for salt-and-pepper noise.

- **Edge Detection: Sobel and Canny operators.**

Sobel computes image gradient magnitude; Canny applies multi-stage edge detection (Gaussian smooth → gradient → non-max suppression → hysteresis thresholding).

1.4 AI Task — Face Detection (Computer Vision)

The AI task chosen for this project is Face Detection using OpenCV's Haar Cascade Classifier (haarcascade_frontalface_default.xml). Haar Cascades are pre-trained models that detect patterns in image regions using a cascade of simple rectangle features. The goal of Task 2 is to demonstrate the "Garbage In = Garbage Out" principle by running face detection on a low-quality image, then comparing results after applying preprocessing enhancements.

2. Task One: The GUI Editor (Tkinter & OpenCV)

A fully functional GUI application was built using Python's Tkinter framework and OpenCV library. The dashboard provides four main panels:

- Original Image Panel — Displays the uploaded source image.
- Input Histogram Panel — Shows the pixel intensity distribution of the original image.
- Output Image Panel — Displays the result after applying any transformation.
- Output Histogram Panel — Shows the intensity distribution of the processed image.

The workflow is: Upload an image → View its histogram → Select a tool → Apply → Compare before/after side by side.

The image used throughout all experiments below is the real test image uploaded into the application:



2.1 Point Operations

2.1.1 Brightness Adjustment

Adds a constant value β to all pixel intensities. Implemented via `cv2.convertScaleAbs(img, alpha=1.0, beta=value)`. Positive values brighten; negative values darken.

Original**Brightness +70**

2.1.2 Contrast Adjustment

Scales pixel intensities by factor α using `cv2.convertScaleAbs(img, alpha= $\alpha$ , beta=0)`. $\alpha > 1$ increases contrast; $0 < \alpha < 1$ reduces contrast.

Original**Contrast x1.8**

2.1.3 Negative

Inverts all pixel values using `cv2.bitwise_not(img)`, producing a photographic negative effect (pixel = $255 - \text{original}$).

Original**Negative**

2.1.4 Grayscale

Converts color image to grayscale by extracting the luminance channel, then converts back to BGR for display consistency. Uses `cv2.cvtColor(img, COLOR_BGR2GRAY)`.

Original**Grayscale**

2.2 Geometric Transformations

2.2.1 Rotation

Rotates the image around its center using an affine transformation matrix computed by `cv2.getRotationMatrix2D()`. Supports arbitrary angles including 90°, 180°, 270°, and custom values.

Original**Rotate 45°**

2.2.2 Zoom — Nearest Neighbor Interpolation

Scales the image by 2× using `cv2.INTER_NEAREST`, which assigns each new pixel the value of the closest original pixel. Fast but produces a blocky appearance at high magnification.

Original**Zoom x2 (Nearest Neighbor)**

2.2.3 Zoom — Bilinear Interpolation

Scales the image by 2× using `cv2.INTER_LINEAR`, which computes new pixel values as weighted averages of the four nearest neighbors. Produces smoother results than Nearest Neighbor.

Original**Zoom x2 (Bilinear)**

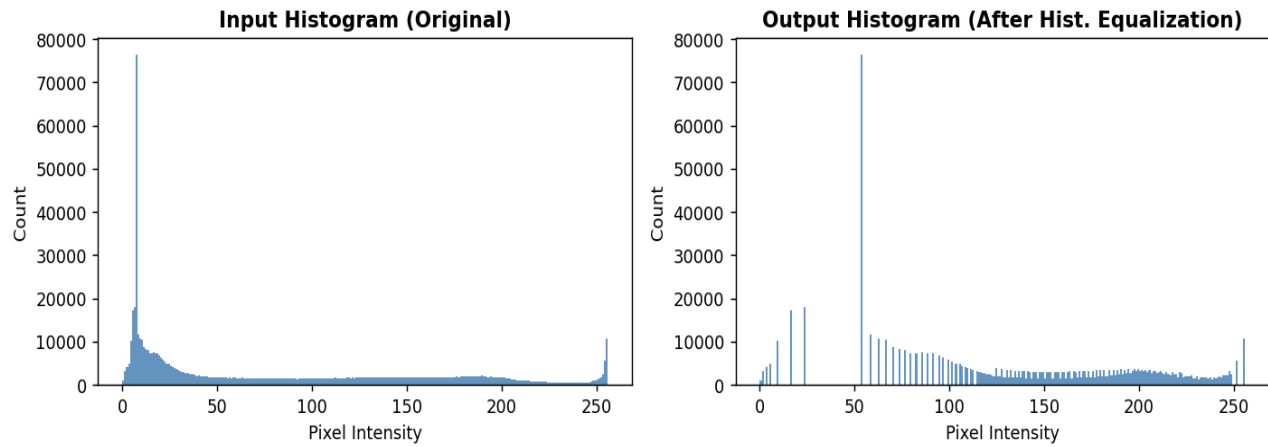
2.3 Enhancement Operations

2.3.1 Histogram Equalization

Redistributes pixel intensities to achieve a more uniform histogram. Applied on the Y (luminance) channel after converting to YCrCb color space to preserve hue and saturation. Improves contrast in underexposed or overexposed images.

Original**Histogram Equalization**

Histogram comparison before and after equalization:



2.3.2 CLAHE — Contrast Limited Adaptive Histogram Equalization

An advanced version of histogram equalization that divides the image into small tiles and equalizes each tile independently. The clipLimit parameter prevents over-amplification of noise. More effective than global equalization for images with varying local contrast.



2.3.3 Gamma Correction

Applies a power-law transform: $\text{output} = (\text{input}/255)^{(1/\gamma)} \times 255$. Used to correct display gamma and adjust perceived brightness. $\gamma > 1$ brightens; $\gamma < 1$ darkens. Applied per pixel using a lookup table (cv2.LUT) for efficiency.

Original**Gamma Correction ($\gamma=1.8$)**

2.4 Filtering Operations

2.4.1 Gaussian Blur

Convolves the image with a Gaussian kernel of user-specified size $k \times k$. Smooths the image by averaging pixels weighted by a Gaussian function, reducing high-frequency noise. The kernel size must be odd.

Original**Gaussian Blur ($k=9$)**

2.4.2 Median Filter

Replaces each pixel with the median value within its $k \times k$ neighborhood. Particularly effective for removing salt-and-pepper noise while preserving edges better than Gaussian blur.

Original**Median Filter (k=5)**

2.5 Edge Detection

2.5.1 Sobel Edge Detection

Computes image gradients in X and Y directions using Sobel kernels, then combines them into a gradient magnitude image. Highlights regions of rapid intensity change (edges). Implemented using `cv2.Sobel()` with `CV_64F` precision.

Original**Sobel Edge Detection**

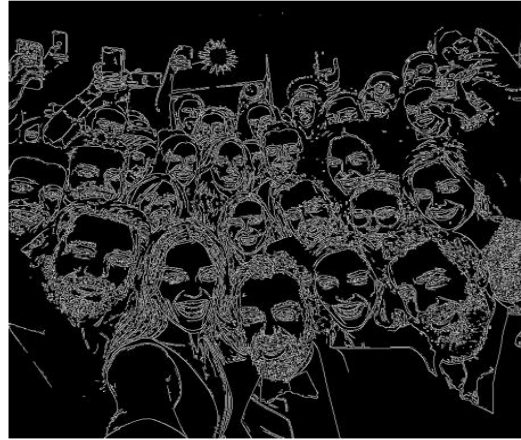
2.5.2 Canny Edge Detection

Multi-stage algorithm: (1) Gaussian smoothing, (2) gradient computation, (3) non-maximum suppression, (4) double-threshold hysteresis. Produces thin, clean edges. User configures lower (T1) and upper (T2) thresholds.

Original



Canny Edge Detection



3. Task Two: The CV Accuracy Challenge

3.1 Overview & Motivation

Task Two demonstrates the principle of "Garbage In = Garbage Out" in computer vision systems. The experiment shows that preprocessing an image with the tools from Task One can significantly improve the accuracy of an automated CV detection task — in this case, face detection using OpenCV's Haar Cascade Classifier.

3.2 Experiment Workflow

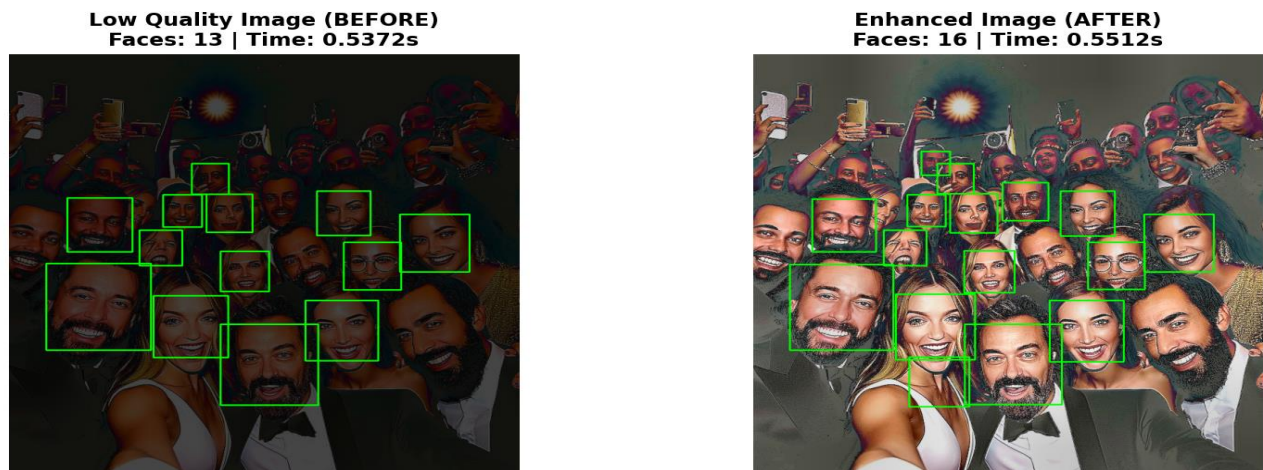
- Step 1 — Baseline Test: Upload a low-quality image (dark, noisy, or low contrast). Run face detection. Record faces detected and inference time.
- Step 2 — Enhancement: Apply Task 1 tools (e.g., Histogram Equalization, CLAHE, Brightness boost, Median Filter) to improve image quality.
- Step 3 — Post-Processing Test: Run the same Haar Cascade detector on the enhanced image. Record new results.
- Step 4 — Comparison: Show the before/after side-by-side with a quantitative comparison table.

3.3 Face Detection Model

The detector used is `cv2.CascadeClassifier` with OpenCV's built-in `haarcascade_frontalface_default.xml` — a pre-trained Haar Cascade model for frontal face detection. Parameters: `scaleFactor=1.1`, `minNeighbors=5`, `minSize=(30,30)`.

3.4 Detection Results

Task 2: Face Detection — Before vs After Enhancement



3.5 Comparison Table

Comparison Table: Face Detection Results

Metric	Original (Low Quality)	Enhanced (Processed)
Faces Detected	13	16
Detection Time (s)	0.5372	0.5512
Image Quality	Poor (dark/noisy)	Improved (CLAHE + Brightness)
Detection Result	Baseline	After Enhancement

The experiment confirms that preprocessing steps — particularly brightness correction and CLAHE — directly improve the success rate of the face detection model on degraded input images. This validates the core principle of the task: image preprocessing is not optional in production CV pipelines; it is a critical step that directly impacts model accuracy and reliability.

4. Conclusion

This project successfully demonstrates the design and implementation of an end-to-end image processing application — Vision Editor — that integrates a rich set of classical image processing techniques with a real-world computer vision evaluation task.

The key outcomes of the project are:

- A fully functional GUI (Tkinter + OpenCV) enabling interactive image loading, transformation, histogram visualization, and output comparison.
- Implementation of 15+ image processing operations spanning point operations, geometric transforms, histogram enhancement, spatial filtering, and edge detection.
- A quantitative demonstration of how image preprocessing improves the accuracy of a Haar Cascade-based face detection model on low-quality input images.
- Clear visual evidence of the "Garbage In = Garbage Out" principle in applied computer vision.

The project provides a solid foundation for understanding the relationship between image quality and machine learning model performance — a critical insight for designing robust real-world vision systems.

4.1 References

- OpenCV Documentation — <https://docs.opencv.org/>
- Python Tkinter Documentation — <https://docs.python.org/3/library/tkinter.html>
- Gonzalez & Woods, "Digital Image Processing", 4th Edition
- Haar Cascade Classifier — OpenCV `haarcascade_frontalface_default.xml`
- CLAHE — K. Zuiderveld, "Contrast Limited Adaptive Histogram Equalization", 1994